Tai lieu chuan bi

TÀI LIỆU CHUẨN BỊ KIẾN THỨC (BẢN CHI TIẾT) — ĐỀ TÀI 14: WiFi MESH DRONE/ROBOT

Bản này tổng hợp thêm nội dung từ ghi chú tay của bạn, đào sâu đều 3 mảng: Lý thuyết mạng Mesh, Firmware ESP32, và Middleware/Dashboard — dựa trên nền bạn đã có: từng làm đồ án nhúng RFID với PIC16F877A, và đã học lý thuyết mạng máy tính nhưng chưa thực hành.

---

0. KIẾN TRÚC TỔNG THỂ — NHÌN TRƯỚC TOÀN BỘ HỆ THỐNG

Trước khi vào chi tiết, cần hình dung rõ hệ thống có 3 lớp tách biệt, mỗi lớp dùng công nghệ khác nhau:

```
[Lớp 1: Firmware — 4 x ESP32-CAM]
   Leaf(Drone) --Mesh(multi-hop)--> Parent(Robot) --Mesh--> Root
                                                              |
                                                        UART/USB (921600 bps)
                                                              |
                                                              v
[Lớp 2: Middleware — Node.js trên PC]
   serialport (đọc byte thô) --> ghép chunk theo frame_id --> socket.io (WebSocket)
                                                              |
                                                              v
[Lớp 3: Frontend — React trên trình duyệt]
   Nhận Uint8Array/Base64 --> hiển thị ảnh + vẽ sơ đồ cây + thông số (Ping/Pin/FPS)
```

Ba lớp này ứng với 3 mảng kiến thức bạn cần chuẩn bị. Đi lần lượt từng phần bên dưới.

PHẦN A — LÝ THUYẾT MẠNG MESH (Networking)

A.1. Vì sao Multi-hop lại “tăng phạm vi” thay vì “tăng tốc độ”?

Trong ghi chú của bạn có ghi đúng trọng tâm: mục tiêu multi-hop là **tăng phạm vi di chuyển**, không phải tăng tốc độ. Đây là điểm hay bị hiểu nhầm nên cần nói rõ:

Một ESP32 phát WiFi công suất thấp có tầm phủ chỉ vài chục mét. Nếu Drone bay ra xa hơn khoảng đó, nó **mất kết nối trực tiếp với Root**. Multi-hop giải quyết bằng cách: Drone không cần “nói chuyện” trực tiếp với Root — nó chỉ cần nằm trong tầm sóng của **một node khác** (ví dụ Robot), rồi node đó tiếp sức giúp. Giống hệt việc bạn hét không tới người ở xa, nhưng có thể nhờ người đứng giữa nhắc lại giúp.

Cái giá phải trả: mỗi lần “nhắc lại” (mỗi hop) đều tốn thời gian xử lý + độ trễ truyền lại, nên **tốc độ thực tế (throughput) giảm dần theo số hop** — đây chính là một trong 3 chỉ số hội đồng chấm (Phần 1, mục 2 tài liệu đề bài gốc).

A.2. Cơ chế Self-healing — giải thích đúng theo kịch bản bạn đã vẽ

Ghi chú của bạn mô tả rất chính xác kịch bản cần đo: *“khi 1 drone ra khỏi vùng của Router chính, thì Drone sẽ kết nối với Drone khác đã có mạng.”* Đây chính là khái niệm **roaming/handover** — thuật ngữ này bạn có thể đã gặp trong lý thuyết mạng di động (khi điện thoại di chuyển giữa các trạm BTS). Cơ chế trong mesh hoạt động tương tự:

1. Mỗi node liên tục đo **RSSI** (cường độ tín hiệu) từ các “hàng xóm” xung quanh nó.
2. Nếu RSSI với node cha hiện tại tụt xuống dưới một ngưỡng (hoặc mất hẳn kết nối), node con hiểu là “đường cũ không còn tốt”.
3. Node con quét lại danh sách hàng xóm, chọn node có RSSI mạnh nhất trong số **đang có đường về Root** — đây gọi là “chuyển vùng” như ghi chú của bạn.
4. Toàn bộ quá trình này chạy theo **timer lặp** (bạn ghi đúng: “Lặp timer”) — nghĩa là không có sự kiện tức thời báo “mất kết nối”, mà node kiểm tra định kỳ (ví dụ mỗi vài giây) trạng thái các liên kết xung quanh. **Chính khoảng thời gian của timer này quyết định tốc độ self-healing** — timer càng ngắn, mạng phục hồi càng nhanh, nhưng tốn nhiều tài nguyên xử lý + băng thông cho việc “hỏi thăm” (heartbeat) hơn. Đây là một trade-off bạn nên đo thực nghiệm và nêu trong báo cáo, không chỉ mô tả lý thuyết suông.

A.3. Metrics đánh giá — đúng như bạn vẽ ở cuối trang 1: đánh giá hệ thống ← kiểm tra băng thông ← độ trễ

Ba đại lượng bạn cần đo và phân biệt rõ (đã nêu ở bản trước, nhắc lại có công thức cụ thể để bạn tính khi thực nghiệm):

- **Throughput thực tế** = (Tổng số byte dữ liệu hữu ích nhận được) / (Thời gian nhận). Đo ở cả trường hợp 1 hop, 2 hop, 3 hop để vẽ biểu đồ suy giảm.

- **Latency (độ trễ)** = Thời điểm Root nhận đủ 1 frame ảnh − Thời điểm Leaf bắt đầu chụp ảnh đó. Muốn đo chính xác, bạn cần đồng bộ thời gian tương đối giữa các node (có thể dùng `millis()` cục bộ + tính chênh lệch round-trip, vì các ESP32 không có đồng hồ chung tuyệt đối).
- **Self-healing time** = Thời điểm node phát hiện mất kết nối cũ → Thời điểm node có đường đi mới hoạt động trở lại (đo bằng cách theo dõi callback `onDroppedConnection()` và `onNewConnection()` trong painlessMesh, ghi log timestamp).

A.4. MTU và Fragmentation — bảng so sánh bạn đã ghi, giải thích thêm ý nghĩa

Ghi chú của bạn có bảng so sánh MTU rất hay, đây là bảng đầy đủ hơn để bạn hiểu vì sao mỗi công nghệ có giới hạn khác nhau:

Công nghệ	MTU/kích thước gói tối đa	Vì sao giới hạn như vậy
WiFi (802.11)	~1500 byte	Kế thừa chuẩn Ethernet để tương thích mạng có dây
4G/5G	~1428–1500 byte	Trừ hao thêm overhead của header đóng gói mạng di động (tunneling)
Bluetooth	~23–512 byte	Ưu tiên tiết kiệm năng lượng, gói nhỏ để giảm rủi ro lỗi truyền và giảm tiêu thụ pin

Ý nghĩa với đồ án của bạn: painlessMesh chạy trên nền WiFi nên gói tối đa lý thuyết gần 1500 byte, nhưng để bài chủ động chọn **chunk 1024 byte** — nhỏ hơn hẳn giới hạn — để chừa khoảng trống cho header giao thức mesh (painlessMesh tự thêm header riêng của nó bên ngoài payload bạn gửi) và giảm rủi ro mất gói khi sóng yếu (gói càng nhỏ, xác suất bị nhiễu làm hỏng toàn bộ gói càng thấp).

Liên hệ với lý thuyết OSI/TCP-IP bạn đã học: đây chính là khái niệm **Fragmentation** ở tầng Network (Layer 3) mà bạn học trong môn mạng máy tính — khi gói IP lớn hơn MTU của một chặng mạng, router phải chia nhỏ nó ra. Điểm khác biệt: IP fragmentation là cơ chế **tự động, trong suốt** ở tầng hệ điều hành mạng; còn trong đồ án này, vì bạn code ở tầng ứng dụng (Layer 7) trên vi điều khiển, bạn phải **tự viết logic phân mảnh này bằng tay** (đó là ý nghĩa của cấu trúc `ImageChunkPacket` với `frame_id/chunk_id/total_chunks`). Nói được sự khác biệt này trong báo cáo sẽ cho thấy bạn hiểu bản chất, không chỉ copy code.

PHẦN B — FIRMWARE ESP32 (đối chiếu với kinh nghiệm PIC16F877A của bạn)

B.1. ESP32 khác PIC16F877A ở điểm nào bạn cần lưu ý nhất

Vì bạn đã quen lập trình nhúng qua đồ án RFID với PIC16F877A, đây là các điểm khác biệt quan trọng nhất khi chuyển sang ESP32 cho đề tài này:

- **Lõi kép (dual-core) + FreeRTOS**: PIC16F877A chạy đơn luồng, code của bạn chạy tuần tự từ trên xuống. ESP32 mặc định chạy **FreeRTOS** — một hệ điều hành thời gian thực nhỏ — cho phép nhiều "task" (tác vụ) chạy song song trên 2 lõi. `painlessMesh` tự sinh ra các task nền để xử lý mạng WiFi liên tục, chạy độc lập với vòng lặp `loop()` chính của bạn. Điều này có nghĩa: code chụp ảnh/gửi ảnh của bạn và code xử lý mạng của thư viện chạy **đồng thời**, không phải tuần tự như PIC — bạn cần cẩn thận với việc truy cập chung tài nguyên (ví dụ buffer ảnh) từ 2 luồng khác nhau.
- **Event-driven thay vì polling thuần**: với PIC, có thể bạn từng dùng cách "hỏi liên tục" (polling) trạng thái một chân hoặc một module (như đọc thẻ RFID). Với ESP32/painlessMesh, mô hình chính là **event-driven**: bạn đăng ký các hàm callback (`onReceive`, `onNewConnection`...) và hệ thống **tự gọi** hàm của bạn khi có sự kiện xảy ra, bạn không cần chủ động hỏi "có gói tin mới chưa?" liên tục. Đây gần với khái niệm ngắt (interrupt) mà bạn đã quen — nhưng là "ngắt phần mềm" ở tầng cao hơn (do RTOS lịch trình), khác với ngắt phần cứng cấp thấp bạn từng cấu hình trực tiếp trên thanh ghi PIC.
- **RAM lớn hơn nhưng vẫn là tài nguyên khan hiếm với ảnh**: ESP32 có RAM nhiều hơn PIC16F877A đáng kể (khoảng vài trăm KB so với vài trăm byte của PIC), nhưng một ảnh JPEG ~30KB cộng với chồng chất buffer mạng của WiFi stack vẫn có thể gây tràn bộ nhớ nếu bạn giữ nhiều ảnh cùng lúc trong RAM. Đây là lý do đề bài nhấn mạnh "tối ưu RAM" — bạn nên giải phóng buffer ảnh (`esp_camera_fb_return()`) ngay sau khi gửi xong, không giữ lại "phòng hờ".

B.2. Thiết kế nguồn điện — vì sao đây là phần dễ bị đánh giá thấp nhưng lại hay gây lỗi nhất

Ghi chú của bạn có nhắc "Thiết kế nguồn điện" như một đầu việc riêng — đúng vậy, đây thường là nguyên nhân gây lỗi khó chẩn đoán nhất khi demo (module tự reset, mất kết nối WiFi giả). Lý do kỹ thuật: module WiFi của ESP32 khi phát sóng (đặc biệt lúc kết nối/gửi burst dữ liệu) có thể **kéo dòng đỉnh (peak current) lên tới 400-500mA trong khoảnh khắc ngắn**, dù dòng trung bình thấp hơn nhiều. Nếu mạch hạ áp LM2596/Mini-360 hoặc pin 18650 không đáp ứng đủ dòng đỉnh này, điện áp sẽ sụt tạm thời, gây treo/reset ESP32 — hiện tượng dễ bị nhầm là "lỗi phần mềm" trong khi thực chất là lỗi nguồn. Bạn nên tìm hiểu thêm về tụ lọc (capacitor) đặt gần chân nguồn ESP32-CAM để giảm hiện tượng này.

PHẦN C — MIDDLEWARE & DASHBOARD (đào sâu theo ghi chú của bạn)

C.1. Vì sao UART giữa Root và PC chạy ở baud rate cao (921600 bps)?

Ghi chú của bạn ghi đúng: Root Node giao tiếp với PC qua UART tốc độ cao — 921600 bps, cao hơn nhiều so với baud rate mặc định

thường thấy (9600 hay 115200). Lý do: dữ liệu ảnh (dù đã fragment) vẫn là luồng byte lớn liên tục chảy từ Root sang PC; nếu dùng baud rate thấp, cổng UART sẽ trở thành **nút thắt cổ chai (bottleneck)** còn tệ hơn cả bản thân mạng mesh không dây — tức là bạn đo throughput/latency của mesh không chính xác vì bị giới hạn bởi UART chứ không phải bởi sóng WiFi. Đây là điểm cần nêu rõ trong báo cáo khi giải thích thiết kế hệ thống.

C.2. WebSocket vs MQTT — khi nào dùng cái nào, và vì sao đề tài này hợp với WebSocket

Ghi chú của bạn có nhắc cả hai — đây là điểm nên làm rõ để không bị hội đồng hỏi vặn "sao không dùng MQTT cho 'chuẩn IoT'":

- **MQTT**: mô hình publish/subscribe qua một **broker trung gian**, rất phù hợp cho hệ thống có nhiều thiết bị IoT phân tán, cần độ tin cậy giao tiếp qua lại linh hoạt (nhiều publisher, nhiều subscriber, thậm chí qua Internet). Yêu cầu phải chạy thêm một broker (ví dụ Mosquitto).
- **WebSocket**: kết nối 2 chiều, **trực tiếp** giữa 1 server (Node.js middleware) và 1 client (trình duyệt), độ trễ thấp hơn vì không phải "đi vòng" qua broker, phù hợp khi bạn chỉ có **một luồng dữ liệu tập trung** (từ Root ra Dashboard) cần cập nhật thời gian thực.

Với kiến trúc đề tài của bạn — chỉ có 1 Root, 1 middleware, 1 dashboard — WebSocket là lựa chọn hợp lý và đơn giản hơn để triển khai trong thời gian ngắn. Nếu muốn "nâng cấp" ở phần hướng phát triển của báo cáo, bạn có thể đề xuất chuyển sang MQTT khi mở rộng hệ thống lên nhiều Root/nhiều dashboard cùng lúc.

C.3. Luồng xử lý dữ liệu ảnh nhị phân (Binary Data pipeline) — đúng như bạn đã vẽ

Ghi chú của bạn mô tả đúng luồng cần cài đặt:

1. Node.js đọc **byte thô** từ cổng Serial (thư viện `serialport`).
2. Ghép các chunk có cùng `frame_id` lại thành buffer đầy đủ — dùng kiểu dữ liệu `Buffer` của Node.js (tương đương khái niệm mảng byte).
3. Chuyển sang dạng gửi được qua WebSocket: hoặc gửi thẳng dạng **binary/Blob** (hiệu quả hơn, khuyến khích dùng vì đỡ tốn băng thông), hoặc encode sang **Base64** (dễ debug, dễ nhúng trực tiếp vào thẻ `<img src="...">` trong React, nhưng tốn thêm khoảng 33% dung lượng so với binary gốc — điều này bạn nên biết để giải thích trade-off nếu hội đồng hỏi vì sao chọn cách nào).
4. Phía React nhận qua `socket.io`, dựng lại thành ảnh hiển thị (nếu nhận Blob thì dùng `URL.createObjectURL()`; nếu Base64 thì gán thẳng vào `src`).

C.4. Dashboard cần hiển thị gì — theo đúng danh sách bạn đã liệt kê

Ghi chú của bạn liệt kê 3 nhóm thông tin dashboard cần có — đây là checklist tính năng frontend nên bám theo: - **Luồng ảnh từ camera** (theo thời gian thực, Motion JPEG 2-5 FPS như đề bài gốc). - **Sơ đồ cây mạng (Tree topology)** — biểu diễn trực quan Root → Parent → Leaf, cập nhật lại ngay khi có sự kiện self-healing (đây là phần "ăn điểm" trực quan lớn khi demo). - **Thông số từng drone**: Ping (độ trễ), Pin (mức năng lượng còn lại — nếu bạn có đo được điện áp pin qua ADC trên ESP32), FPS (số khung hình/giây thực tế nhận được) — đây chính là các con số minh chứng cho 3 chỉ số throughput/latency/self-healing một cách trực quan, sinh động ngay trên giao diện, thay vì chỉ có trong file log.

BẢNG THUẬT NGỮ MỞ RỘNG

Thuật ngữ	Giải thích
Roaming / Handover	Node chuyển từ kết nối với node cha cũ sang node cha mới khi tín hiệu cũ yếu đi
Heartbeat	Tín hiệu "còn sống" gửi định kỳ để các node biết nhau vẫn còn kết nối
FreeRTOS	Hệ điều hành thời gian thực nhỏ chạy nền trên ESP32, quản lý nhiều task song song
Task (RTOS)	Một luồng xử lý độc lập được RTOS lập lịch chạy, khác với vòng lặp `loop()` tuần tự
Peak current (dòng đỉnh)	Dòng điện tăng vọt trong khoảnh khắc ngắn, thường lúc module WiFi phát sóng
Broker (MQTT)	Máy chủ trung gian nhận/phân phối lại tin nhắn giữa các thiết bị publish/subscribe
Blob	Kiểu dữ liệu nhị phân thô trong JavaScript, dùng để truyền ảnh hiệu quả hơn Base64

CÂU HỎI HỘI ĐỒNG MỞ RỘNG (thêm dựa trên phần bạn vừa bổ sung)

1. Vì sao chọn baud rate UART 921600 thay vì mặc định 115200? Nếu để 115200 thì điều gì sẽ xảy ra với kết quả đo throughput?

2. Cơ chế self-healing dựa trên timer lặp — nếu timer quá ngắn thì có hại gì cho mạng không? (gợi ý: tốn băng thông cho heartbeat, có thể gây "rung lắc" đổi node cha liên tục nếu tín hiệu dao động nhẹ quanh ngưỡng — hiện tượng gọi là "flapping")
3. Vì sao không dùng MQTT như chuẩn IoT phổ biến mà lại chọn WebSocket?
4. Gửi ảnh dạng Base64 hay Binary/Blob qua WebSocket, bạn chọn cái nào và đánh đổi ra sao?
5. Nếu ESP32-CAM bị treo/reset ngẫu nhiên lúc phát WiFi mạnh, nguyên nhân phần cứng nào cần loại trừ trước khi nghi ngờ do phần mềm?

GỢI Ý HỌC THEO TUẦN (cập nhật, gắn với nền tảng bạn đã có)

- **Trước Tuần 1**: vì bạn đã quen structs/con trỏ từ đồ án PIC, phần cần bù thêm là khái niệm **task/RTOS và event-driven của ESP32** — nên đọc nhanh phần "FreeRTOS basics" trong tài liệu Espressif trước khi đọc painlessMesh, sẽ dễ hiểu vì sao thư viện hoạt động "ngầm" song song với code của bạn.
- **Trước Tuần 3**: vì lý thuyết mạng bạn mới học chưa thực hành — nên tự tay dựng lại ví dụ đơn giản: 2 ESP32 gửi qua lại một chuỗi text dài giả lập vượt quá 1024 byte, tự tay code phân mảnh trước, để cảm nhận vấn đề trước khi ghép logic camera vào (tránh học 2 cái khó cùng lúc: vừa camera vừa fragmentation).
- **Trước Tuần 6**: thử nghiệm nhỏ so sánh gửi ảnh test qua Base64 và qua Blob nhị phân trên cùng 1 mạng, đo thử độ trễ chênh lệch — có số liệu thật để trả lời câu hỏi hội đồng số 4 ở trên.
- **Trước Tuần 8**: chuẩn bị kịch bản đo self-healing rõ ràng và lặp lại được (ví dụ: cố định vị trí 3 node, tắt nguồn node giữa ở một thời điểm đánh dấu, đo bằng log timestamp của callback) — tránh đo kiểu "cảm tính bằng mắt" sẽ không thuyết phục hội đồng.

So với bản đầu tiên, bản này đi sâu thêm vào: cơ chế roaming/self-healing dựa trên timer, đối chiếu kiến trúc PIC vs ESP32 (RTOS, event-driven, dòng đỉnh nguồn điện), so sánh WebSocket/MQTT, và luồng xử lý ảnh nhị phân đầy đủ từ Serial đến trình duyệt — đúng theo 3 mảng bạn đã ghi chú tay và yêu cầu đào sâu đều.